EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking the answer.

We must write a loop before we can display a 3×3 board in C#.

How confident are you? Circle one: Confident / Getting there / Need help

Explain your choice, identifying one concept you can now use independently.

GREEN PEN — Correct and explain

Choose one mistake from your independent work. Write the correct method and explain precisely why your original answer was wrong.

Next step: revisit the model or a relevant revision-guide section if needed.

DO NOW — RECALL

Noughts & Crosses · Part 1 — Sequence before loops

Name: _____ Class: _____ Date: _____

1. In a zero-indexed C# array, which element does array[2] access?

2. Name the three parts of a C# for loop header used to visit every array element.

3. In a C# 2D array organised as rows and columns, what does GetLength(0) return?

4. If a board has 3 rows and 3 columns, how many values does it contain?

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

The item at index 2 — the third item.

2. Model answer

A counter, condition and update.

3. Model answer

The number of rows.

4. Model answer

9.

By the end, you can…

- display a nine-square board using simple sequential C# statements;
- store the nine values in a two-dimensional array;
- access squares by row and column, and spot repeated instructions.

Vocabulary — know these terms

Recall the term — sequence

Recall the term — statement

Recall the term — output

Recall the term — 2D array

Recall the term — row

Recall the term — column

Recall the term — index

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 3

Create a 2D array
Declare string[,] board containing strings "1" through "9". Assign board[1,1] = "X"; .

Question 4

Print the array without loops
Using three separate Console.WriteLine() statements and explicit board[row,col] accesses, display all three rows. Highlight the parts of the code that repeat.

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations, a trace or justified reasoning as appropriate.

Question 1

Create a static board
In .NET Fiddle, write sequential Console.WriteLine() statements to display the nine squares. Do not use a loop.

Question 2

Make a move by editing text
Change the centre displayed square from 5 to X by modifying one of your existing Console.WriteLine() statements.

NEW LEARNING

Read, annotate and explain

A board with five sequential statements

```
Console.WriteLine("1 | 2 | 3");
Console.WriteLine("--+---+--");
Console.WriteLine("4 | 5 | 6");
Console.WriteLine("--+---+--");
Console.WriteLine("7 | 8 | 9");
```

Checkpoint 1: First make something that runs. Next separate game data from the lines that display it. Loops come in Lesson 014.

Start with sequence

Write the five lines in the left panel into a C# console program. Run it. There are no loops ; each statement executes in order.

Make one simple change

Change the middle square 5 to X. Which exact line do you edit? This works — but changing the text every time a player moves is awkward.

Store the squares as data

Put the nine square values into a 2D array. The first index chooses the row; the second chooses the column.

Try board[1,1] = "X"; : the data changes without rewriting every display instruction.

Display the data directly

Write the first row explicitly using three array accesses. Then do the same for rows 1 and 2.

Notice: almost the same code is needed for every row. We will use that repetition to discover loops in Lesson 014.

Explain one step in your own words, using the correct technical terms.

I DO — WORKED EXAMPLE

Follow the worked method

Make the board display before improving it

Create a working output

Run these instructions in order. The result is a 3×3 numbered board.

```
Console.WriteLine("1 | 2 | 3");
Console.WriteLine("--+---+--");
Console.WriteLine("4 | 5 | 6");
Console.WriteLine("--+---+--");
Console.WriteLine("7 | 8 | 9");
```

Change one square

Change the centre of the printed board to X. This shows the limitation of storing positions inside string literals.

```
Console.WriteLine("4 | X | 6");
```

Move the values into a 2D array

```
string[,] board = {
    { "1", "2", "3" },
    { "4", "5", "6" },
    { "7", "8", "9" }
};
```

Square 5 is board[1,1] , not board[5] , because our board uses zero-based row and column indexes.

Print the same values without a loop

```
Console.WriteLine(board[0,0] + " | " + board[0,1] + " | " + board[0,2]);
Console.WriteLine(board[1,0] + " | " + board[1,1] + " | " + board[1,2]);
Console.WriteLine(board[2,0] + " | " + board[2,1] + " | " + board[2,2]);
```

This code works. But how many times do the row-printing instructions repeat? What changes between lines? We'll tackle that in Lesson 014.

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

1. Display three rows

Without loops, write three C# statements that print 1 | 2 | 3, then 4 | 5 | 6, then 7 | 8 | 9.

Method / working:

2. Find the square

With zero-based indexes, what is the row/column index of the centre square (5)?

Method / working:

3. Change the stored data

After declaring a 3×3 string array, write one assignment that places X in the centre.

Method / working:

4. Spot repeated code

Look at three separate row-print statements. What is identical, and what changes each row?

Method / working: